Based on your paper, ***Comprehensive Evaluation and Architectural Optimization of the KDA Hybrid Transformer for Leela Chess Zero on Integrated Hardware***, your general trajectory is **directionally solid and well-reasoned**, but there are critical mathematical, spatial, and numerical pitfalls in the current formulation that need fixing.
Below is an analysis of where your research is on point versus where mistakes or potential oversights lie.

What You Got Right (Right Direction)

- **Policy Head Isolation (Sparing)**: Keeping full $O(T^2)$ pairwise attention for the policy head while hybridizing the backbone encoder is 100% correct. Replacing policy logit generation with linear recurrence would destroy the 64×64 source-to-destination move spatial structure required by LCZero.
- **3:1 Hybrid Architecture**: The 3 KDA to 1 MHA block interleaving mirrors proven configurations (like Kimi Linear). In a chess engine context, 3 KDA layers handle rapid local spatial updates, and the 4th MHA layer acts as a global communication bridge across all 64 squares.
- **Shared Local Memory (SLM) Optimization**: For your Tiger Lake Intel Iris Xe target, caching the 32×32 state matrix (S_t) directly into 4 KiB of SLM per workgroup during the 64-step scan is the optimal hardware path to bypass memory bandwidth bottlenecks.
- **Resetting $S_0 = 0$ on MCTS Evaluations**: Correctly identifying that state persistence across search evaluations will corrupt Monte Carlo Tree Search graph non-deterministically.

⚠️ Critical Mistakes & Gaps to Address

1. 1D Causal Convolutions & Rank-Boundary Artifacts

- **The Problem**: Standard 1D convolutions on a flattened 64-length sequence treat square h1 (index 7) and a2 (index 8) as adjacent neighbors.
- **The Fix**: Before applying spatial convolutions or recurrence inputs, you must reshape the 1D sequence tensor $x \in \mathbb{R}^{B \times 64 \times D}$ into an explicit 2D grid $\mathbb{R}^{B \times 8 \times 8 \times D}$ and use DepthwiseConv2D(3x3) with zero-padding before flattening back to 64 tokens.

2. Multi-Axis Scanning Directional Bias

- **The Problem**: A standard 4-directional scan (orthogonal ranks and files) ignores diagonal tactical relationships (bishops, queens, pawns). A threat moving along a diagonal would require multiple layer steps to propagate through orthogonal scans.
- **The Fix**: Expand the 4-directional scan to an **8-Directional Scan** (D_4 Dihedral group symmetries). Include the 4 main/anti-diagonal traversals alongside the orthogonal ones, assigning 2 heads per direction across 16 total heads.

3. Mixed-Precision Underflow in State Updates

- **The Problem**: Exponentiating g_t ($e^{\exp(g_t)}$) inside an FP16 pipeline during long sequence iterations can cause subnormal underflow or NaN gradients on Iris Xe execution units.

- **The Fix**: FP16 math should only be used for input projections ($q_t, k_t, v_t, \beta_t, g_t$) and final gating. Promoted computations for state matrix updates (S_t) and decay exponentiation **must strictly remain in FP32**.

Architecture Validation Summary

Feature / Component	Baseline Hybrid (Variant C)	Proposed Refinement (Variant D)	Evaluation / Correctness
Backbone Ratio	3 KDA : 1 MHA	3 KDA : 1 MHA	✅ Correct
Policy Head	Full Pairwise MHA	Full Pairwise MHA	✅ Correct
Spatial Scan	4-Directional Orthogonal	8-Directional Dihedral (D_4)	⚠️ Needs Upgrade to 8-Direction
Spatial Convolutions	1D Causal	2D Depthwise (3×3)	❌ Fix 1D Rank-Boundary Leakage
State Recurrence	FP16/FP32 Unspecified	Mixed-Precision (FP32 State)	⚠️ Enforce FP32 for S_t Accumulation

Recommendations for Your Repository

1. **Verify tf.scan behavior in Python**: Before compiling SYCL kernels, ensure your TensorFlow implementation enforces 2D depthwise spatial convolutions prior to directional splitting.
2. **Check SYCL SLM Allocation**: In kda_fused_recurrence_kernel, verify that state[32][32] is explicitly barrier-synced (group_barrier) before and after the 64-step recurrence loop to prevent race conditions across threads within a workgroup.